EXPERIMENT NO. 13

Title

Letter Combinations of a Phone Number Using Java

Objective

To write a Java program to find all possible letter combinations corresponding to a string of digits from 2 to 9 using the Collection Framework.

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse

Software Requirements

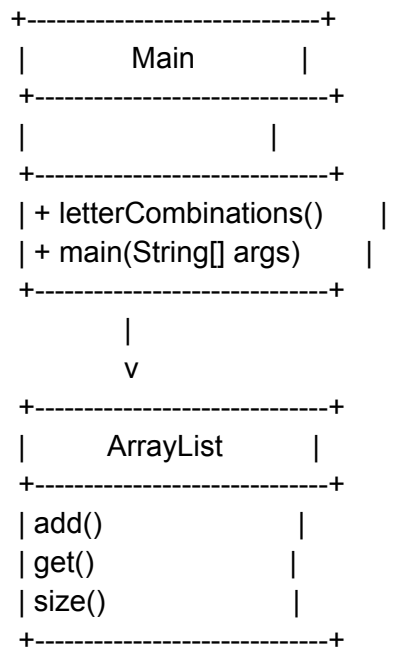
- Java JDK
- VS Code / Eclipse / IntelliJ IDEA
- Windows/Linux operating system

Algorithm / Logic Description

1. Start the program.
2. Read a string containing digits from the user.
3. Create a phone mapping array that maps each digit to its corresponding letters.
4. Create an empty `ArrayList` to store the combinations.
5. If the input string is empty, return the empty list.
6. Initialize the list with an empty string.
7. Traverse each digit in the input string.
8. Get the letters corresponding to the current digit.
9. Create a temporary `ArrayList` to store the new combinations.
10. For each existing combination, append every letter corresponding to the current digit.
11. Update the result list with the newly generated combinations.
12. Display all possible letter combinations.

13. Stop the program.

UML Class Diagram



Source Code

```
import java.util.*;

public class Main {
    static String[] phone = {
        "", "", "abc", "def", "ghi",
        "jkl", "mno", "pqrs", "tuv", "wxyz"
    };

    static List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();

        if (digits.length() == 0)
            return result;

        result.add("");

        for (int i = 0; i < digits.length(); i++) {
            String letters = phone[digits.charAt(i) - '0'];
            List<String> temp = new ArrayList<>();
```

```
            for (String s : result) {
                for (char ch : letters.toCharArray()) {
                    temp.add(s + ch);
                }
            }

            result = temp;
        }

        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter digits: ");
        String digits = sc.nextLine();

        System.out.println(letterCombinations(digits));

        sc.close();
    }
}
```

Compilation and Execution Commands

Compile:

javac Main.java

Execute:

java Main

Output Screen

Example 1: Letter combinations for two digits

Enter digits: 23
[ad, ae, af, bd, be, bf, cd, ce, cf]

Example 2: Letter combinations for a single digit

Enter digits: 2

[a, b, c]

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week13>  & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:50065' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\navee\AppData\Roaming\Code\User\workspaceStorage\436ce91fb1b487e064df7e11362fd885\redhat.java\jdt_ws\Week13_2b8867f4\bin' 'Week13'
Enter digits: as
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 49 out of bounds for length 10
        at Week13.letterCombinations(Week13.java:21)
        at Week13.main(Week13.java:42)
PS C:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week13>  c:; cd 'c:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week13'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:50070' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\navee\AppData\Roaming\Code\User\workspaceStorage\436ce91fb1b487e064df7e11362fd885\redhat.java\jdt_ws\Week13_2b8867f4\bin' 'Week13'
Enter digits: 22
[aa, ab, ac, ba, bb, bc, ca, cb, cc]
```

Result

The Java program was successfully implemented to find all possible letter combinations of a phone number using the Collection Framework and the `ArrayList` class.